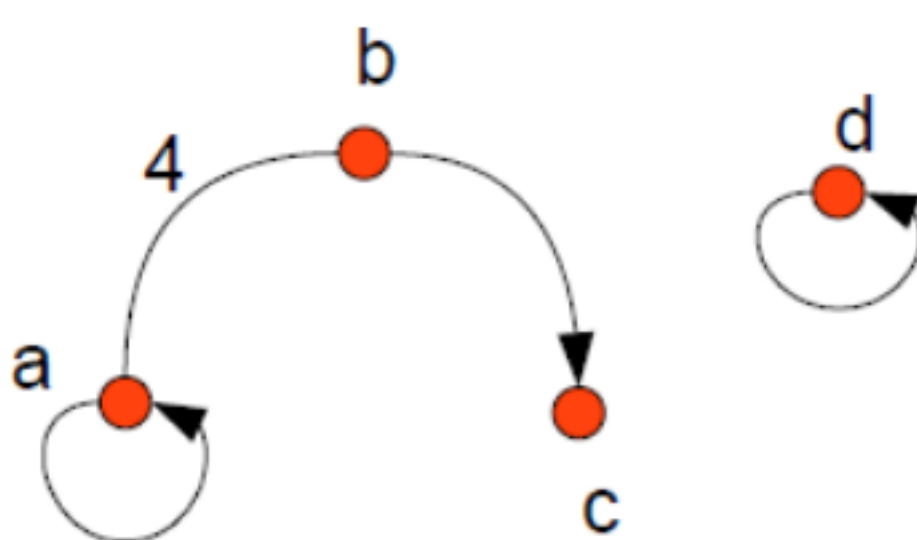


GrafosCamino mínimoGrafo Definición

Dupla compuesta por un conjunto no vacío de vértices V ; un conjunto de aristas que vinculan pares de esos vértices.

Las aristas pueden ser dirigidas o no dirigidas.

$$G = (V; A) \quad V = \{a; b; c; d\} \quad A = \{(a; b); (b; c), (a; a); (d; d)\}$$

Definiciones

Vértice aislado: sin relación (mediante aristas) con otro vértice.

Lazo: arista cuyo vértice origen y destino coincide.

Arista ponderada: a las aristas se les puede asociar un valor representativo de la relación que representan, en este caso podría representar la cantidad de km entre la ciudad a y b.

Peso de una arista: es el valor asociado a una arista ponderada.

Vértices adyacentes (v, w) : v y w son adyacentes si están relacionadas.

Incidencia (v) : conjunto de aristas finalizadas / comenzadas en v .

Incidencia de entrada (v) : conjunto de aristas dirigidas finalizadas en v .

Incidencia de salida (v) : conjunto de aristas dirigidas comenzadas en v .

Adyacencia (v) : conjunto de vértices relacionados con v .

Adyacencia de entrada (v) : vértices iniciales de la incidencia de entrada (v) .

Adyacencia de salida (v) : vértices finales de la incidencia de salida (v) .

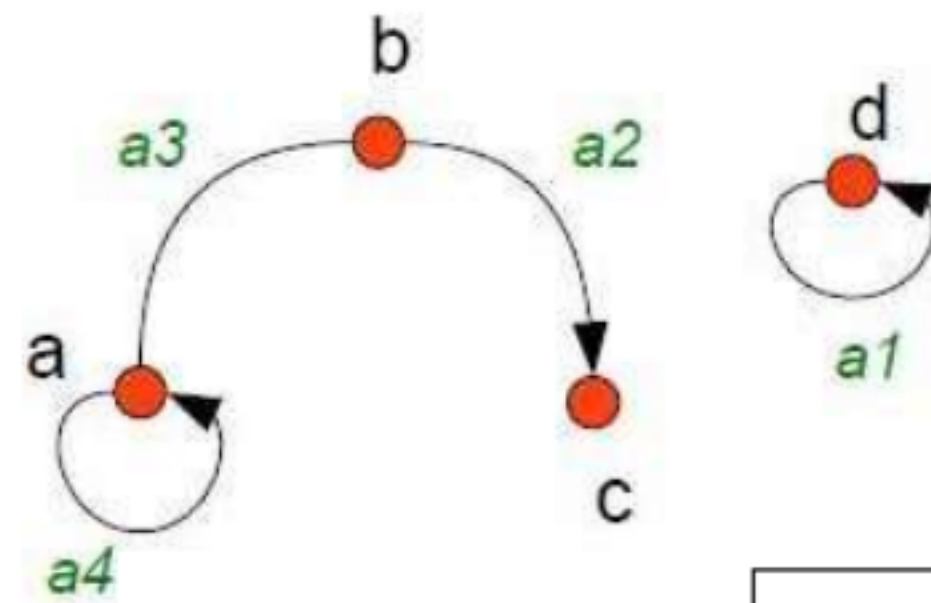
Grado (v) : Cantidad de ocurrencias de v en el conjunto de aristas.

Grado de entrada (v) : Cantidad de aristas dirigidas finalizadas en v .

Grado de salida (v) : Cantidad de aristas dirigidas comenzadas en v .

Fuente: vértice cuyo grado de entrada es 0 y no es aislado.

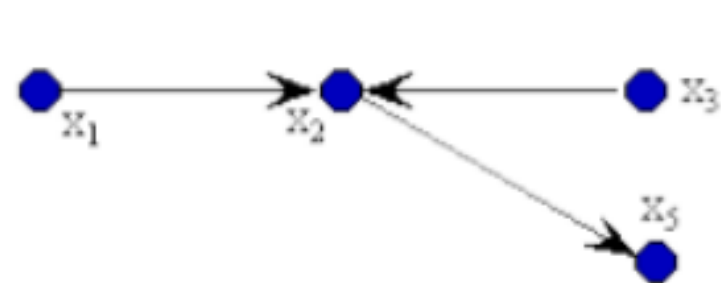
Sumidero: vértice cuyo grado de salida es 0 y no es aislado.



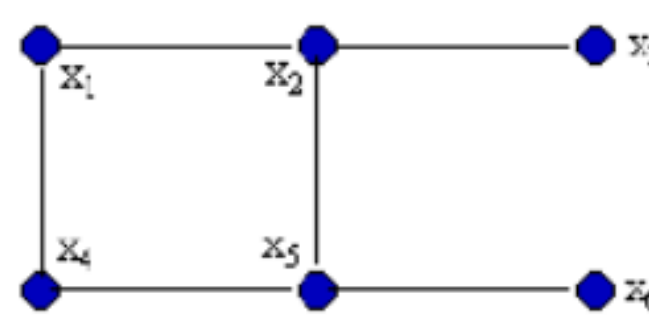
$$A = \{ \{a; b\}; \{b; c\}; \{a; a\}; \{d; d\} \}$$

	a	b	c	d
Adyacencia	$\{a, b\}$	$\{a, c\}$	$\{b\}$	$\{d\}$
Adyacencia de Entrada	$\{a\}$	$\{\}$	$\{b\}$	$\{d\}$
Adyacencia de Salida	$\{a\}$	$\{c\}$	$\{\}$	$\{d\}$
Incidencia	$\{a3, a4\}$	$\{a3, a2\}$	$\{a2\}$	$\{a1\}$
Incidencia de Entrada	$\{a4\}$	$\{\}$	$\{a2\}$	$\{a1\}$
Incidencia de Salida	$\{a4\}$	$\{a2\}$	$\{\}$	$\{a1\}$
Grado	3	2	1	2
Grado de Entrada	1	0	1	1
Grado de Salida	1	1	0	1

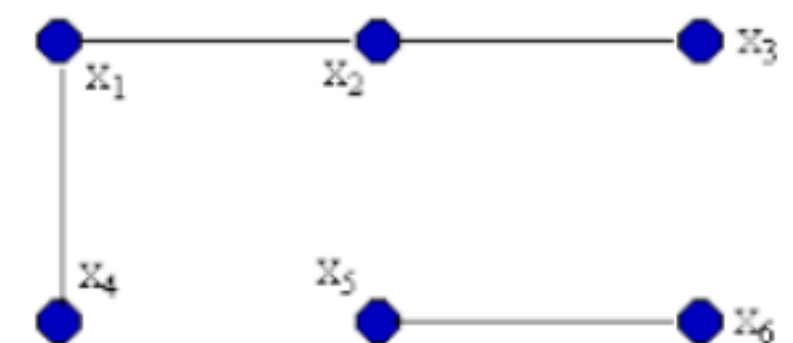
Grafos: Tipos



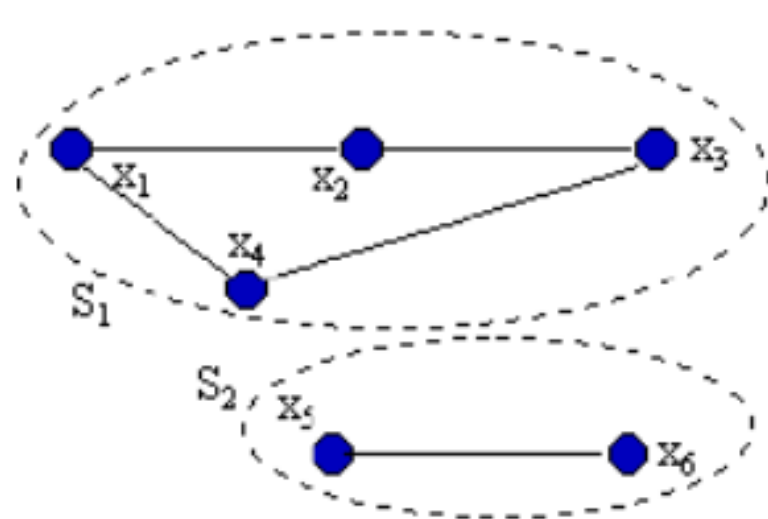
Conexo dirigido



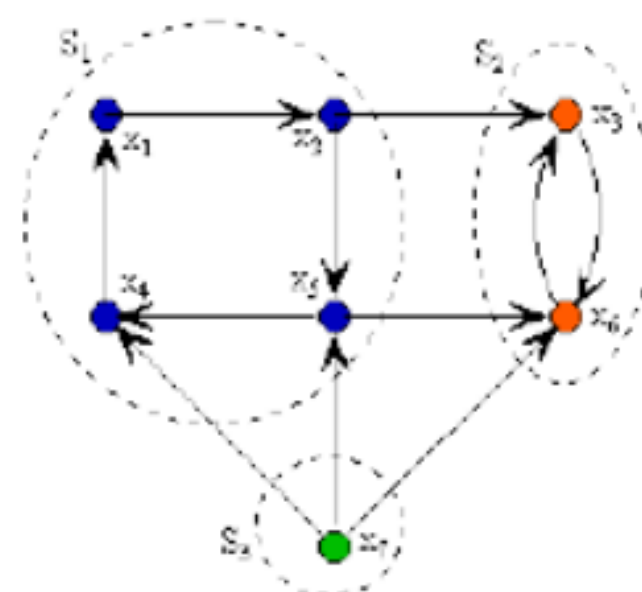
Conexo



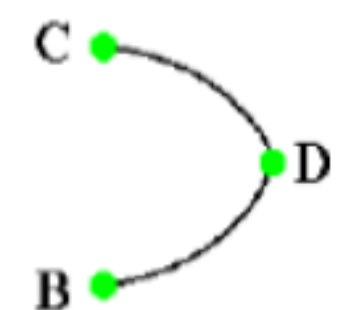
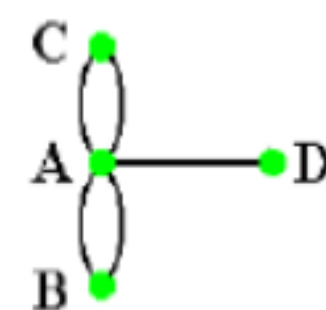
Disconexo



Componentes Conexas



Componentes fuertemente conexas



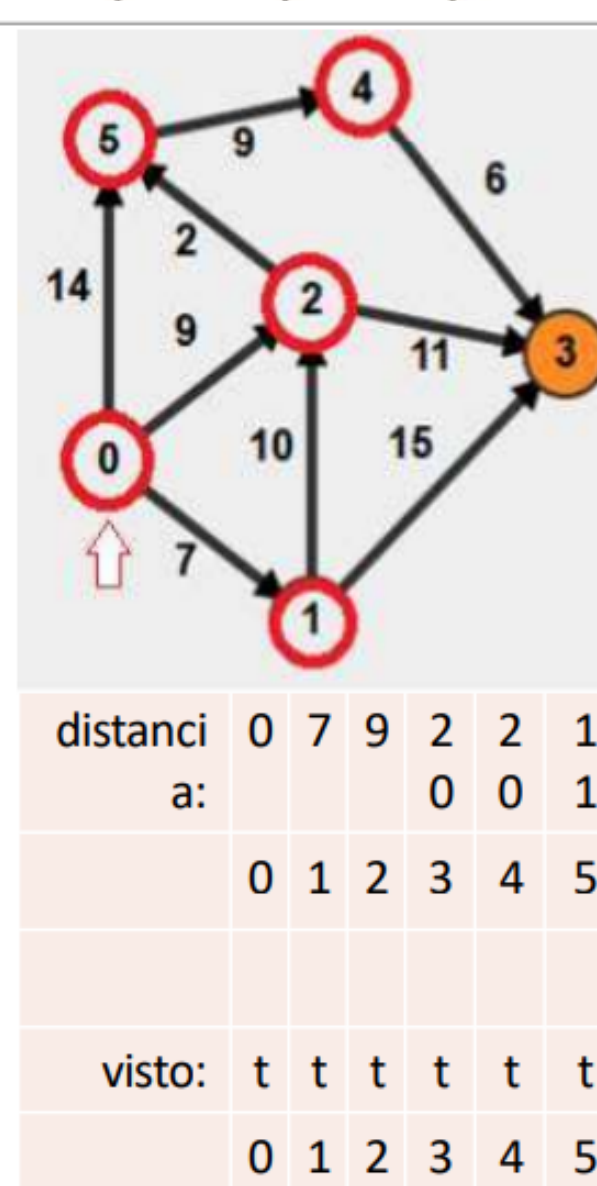
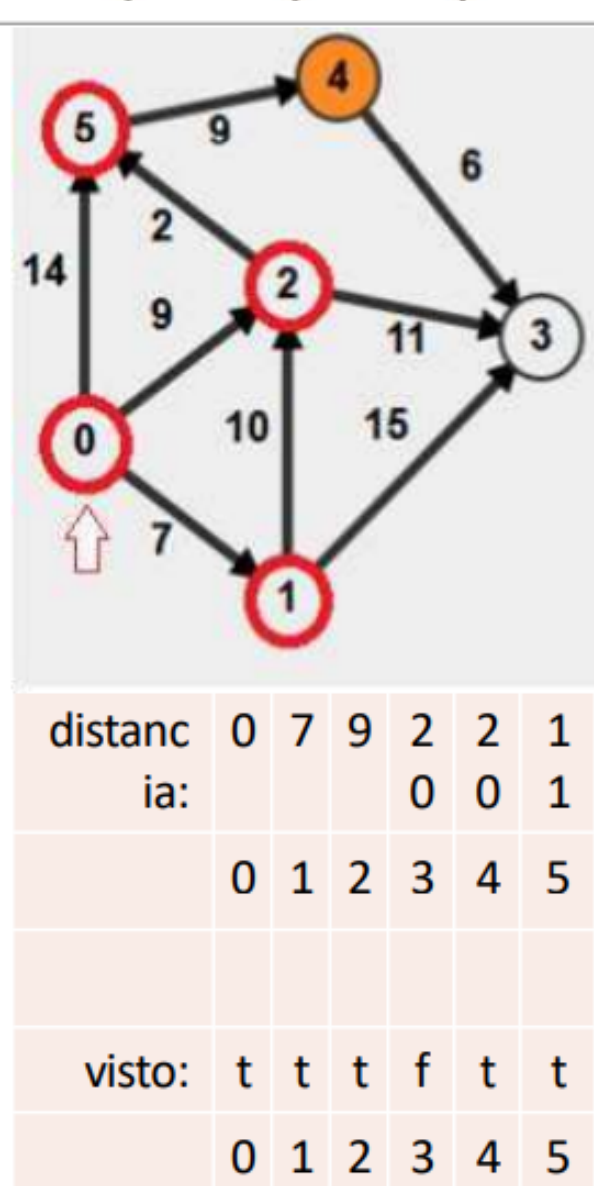
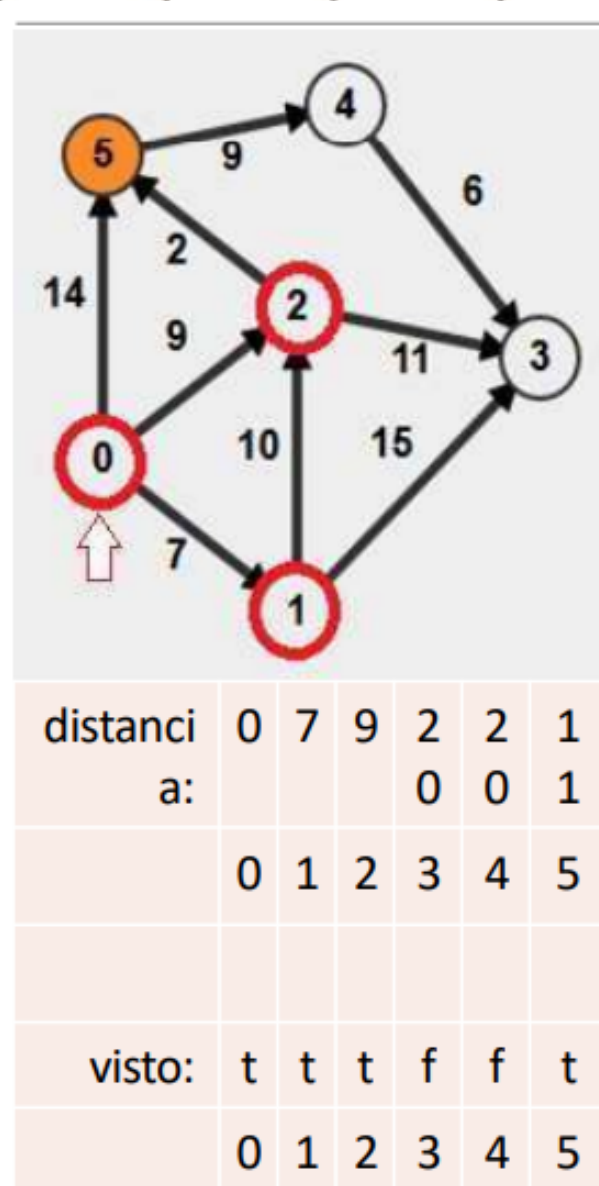
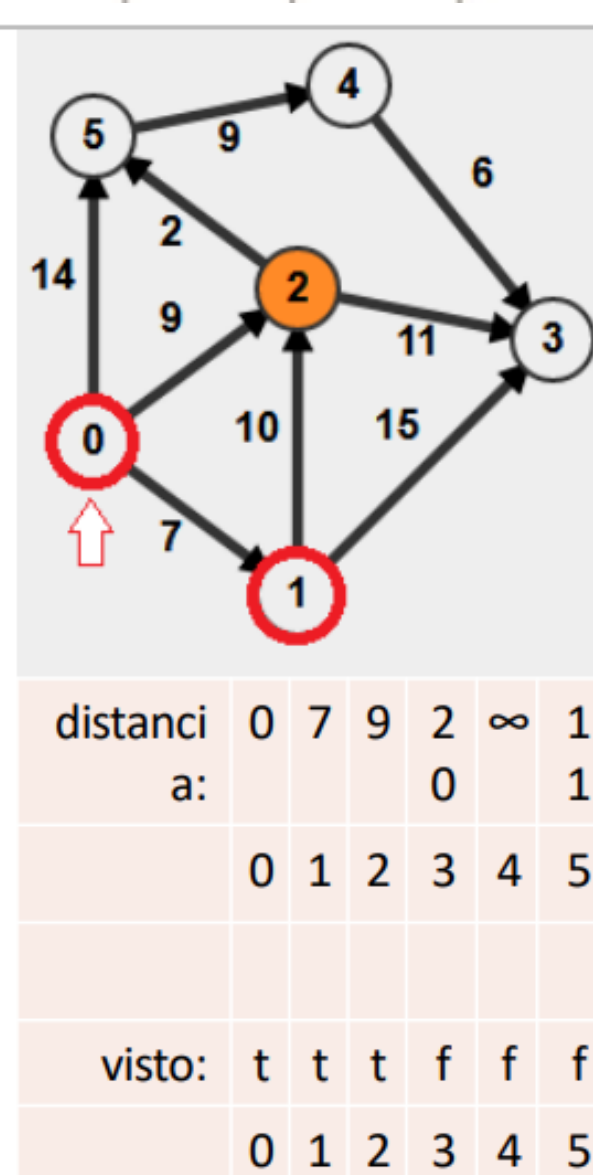
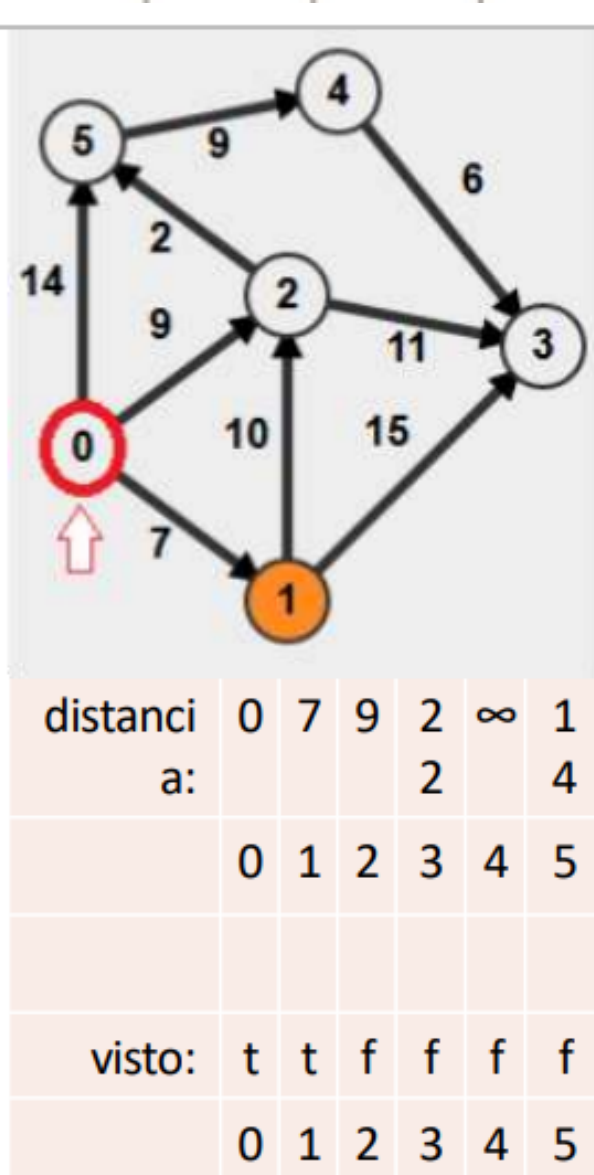
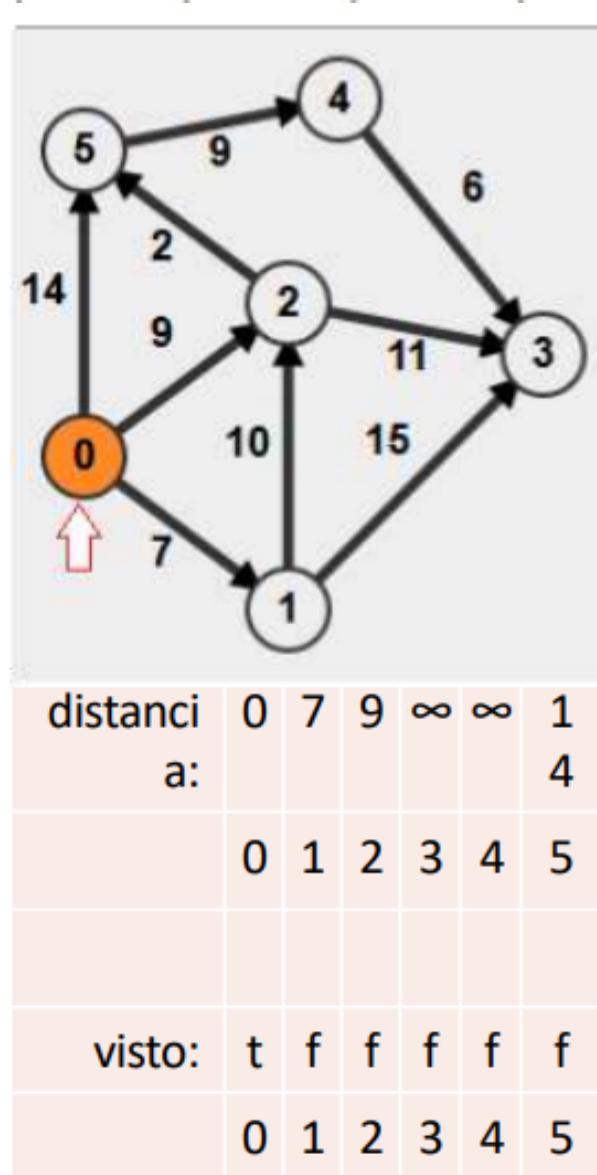
Subgrafo

Dijkstra

Es un algoritmo para la determinación del camino mínimo, **dado un vértice origen, hacia el resto de los vértices** en un grafo que tiene pesos en cada arista.

Consiste en ir explorando todos los caminos más cortos que parten del vértice origen y que llevan a todos los demás vértices.

Cuando se obtiene el camino más corto desde el vértice origen hasta el resto de los vértices que componen el grafo, el algoritmo se detiene.



Dijkstra

```

función Dijkstra (Grafo G, nodo_inicial s)
    entero distancia[n]
    booleano visto[n]
    para cada w ∈ V[G] hacer
        Si (no existe arista entre s y w) entonces
            distancia[w] = Infinito
        Si_no
            distancia[w] = peso (s, w)
    fin si
fin para
distancia[s] = 0
visto[s] = true

mientras que (no_estén_vistos_todos) hacer
    vértice = tomar_el_mínimo_del_vector distancia y que no esté
    visto;
    visto[vértice] = true;
    para cada w ∈ sucesores (G, vértice) hacer
        si distancia[w] > distancia[vértice] + peso (vértice, w)
        entonces
            distancia[w] = distancia[vértice] + peso (vértice, w)
        fin si
    fin para
fin mientras
fin función.

```

Dijkstra (con cola de prioridad)

```

DIJKSTRA (Grafo G, nodo_fuente s)
    para u ∈ V[G] hacer distancia[u] = INFINITO padre[u] = NULL
    visto[u] = false

    distancia[s] = 0
    adicionar (cola, (s, distancia[s])) mientras que cola no es vacía
    hacer
        u = extraer_mínimo(cola)
        visto[u] = true
        para todos v ∈ adyacencia[u] hacer si ¬ visto[v]
            si distancia[v] > distancia[u] + peso (u, v) hacer
                distancia[v] = distancia[u] + peso (u, v) padre[v]
                = u
            adicionar (cola, (v, distancia[v]))

```

Floyd - Warshall

Es otro algoritmo para encontrar el camino mínimo en grafos dirigidos ponderados.

Encuentra el camino entre todos los pares de vértices en una única ejecución.

Compara todos los posibles caminos a través del grafo entre cada par de vértices.

Para que haya una coherencia numérica, supone que no hay ciclos negativos.

Algoritmo:

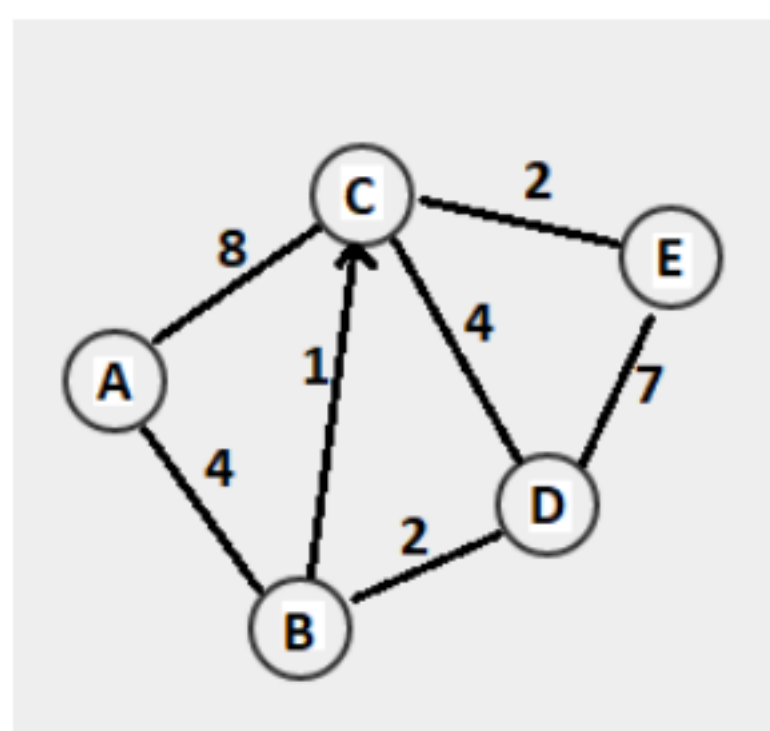
- 1) Formar las matrices iniciales D y R , donde D es la matriz de adyacencia o distancias, y R es una matriz de recorridos del mismo tamaño vacía.
- 2) Se toma $k=1$
- 3) Se selecciona la fila y la columna k de la matriz D y entonces, para i y j , con $i \neq k$, $j \neq k$ e $i \neq j$, hacemos:

$$\bullet \text{ Si } (D_{ik} + D_{kj}) < D_{ij} \longrightarrow D_{ij} = D_{ik} + D_{kj} \text{ y } R_{ij} = R_{kj}$$

En caso contrario, dejamos las matrices como están.

- Si $k \leq n$, aumentamos k en una unidad y repetimos el Paso anterior, en caso contrario paramos las interacciones.

La matriz final D contiene los costos óptimos para ir de un vértice a otro, mientras que la matriz R contiene los penúltimos vértices de los caminos óptimos que unen dos vértices.



iteracion0

distancias:

	A	B	C	D	E
A	0	4	8	∞	∞
B	4	0	1	2	∞
C	8	∞	0	4	2
D	∞	2	4	0	7
E	∞	∞	2	7	0

recorridos:

	A	B	C	D	E
A	-	B	C	D	E
B	A	-	C	D	E
C	A	B	-	D	E
D	A	B	C	-	E
E	A	B	C	D	-

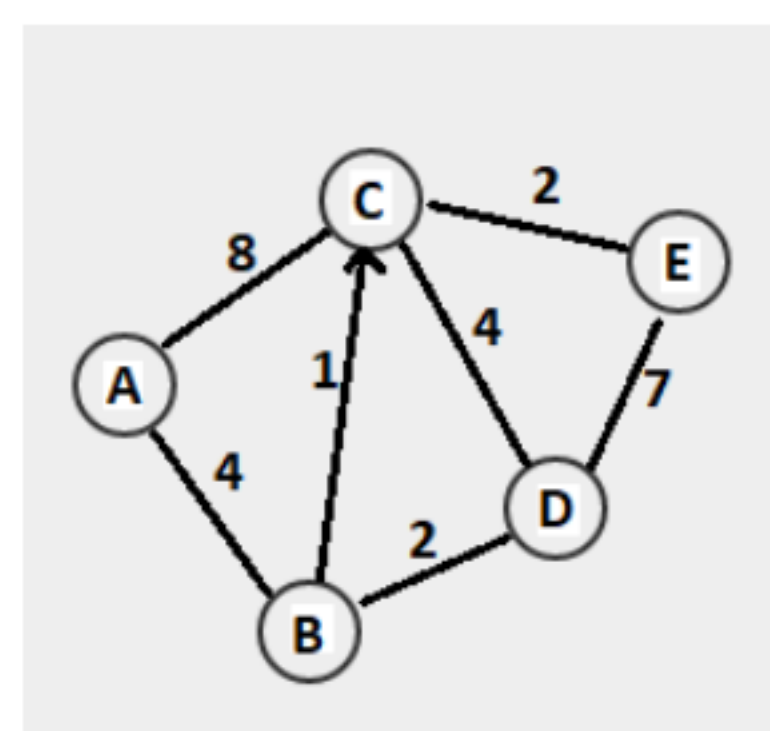
iteracion1

distancias:

	A	B	C	D	E
A	0	4	8	∞	∞
B	4	0	1	2	∞
C	8	12	0	4	2
D	∞	2	4	0	7
E	∞	∞	2	7	0

recorridos:

	A	B	C	D	E
A	-	B	C	D	E
B	A	-	C	D	E
C	A	A	-	D	E
D	A	B	C	-	E
E	A	B	C	D	-



iteracion1

distancias:

	A	B	C	D	E
A	0	4	8	∞	∞
B	4	0	1	2	∞
C	8	12	0	4	2
D	∞	2	4	0	7
E	∞	∞	2	7	0

recorridos:

	A	B	C	D	E
A	-	B	C	D	E
B	A	-	C	D	E
C	A	A	-	D	E
D	A	B	C	-	E
E	A	B	C	D	-

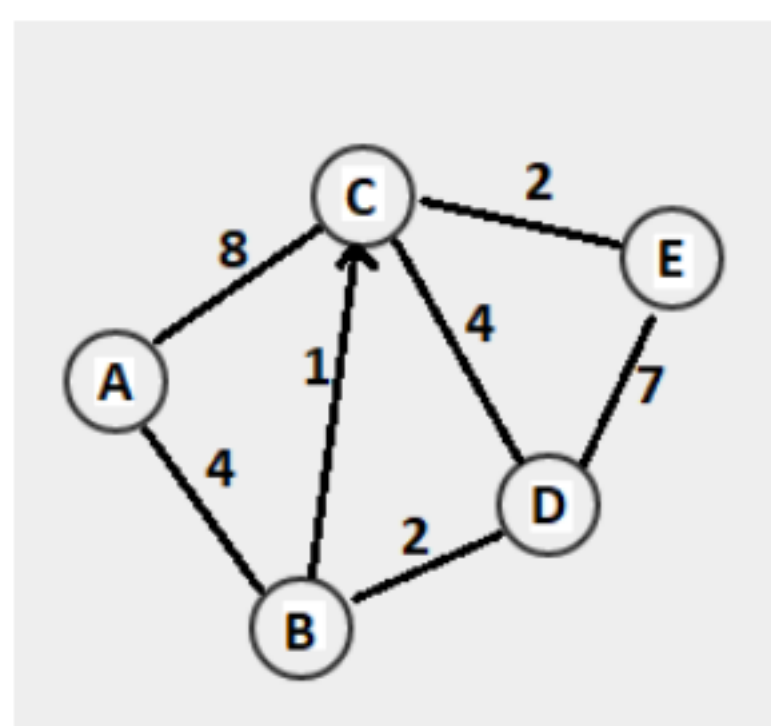
iteracion2

distancias:

	A	B	C	D	E
A	0	4	5	6	∞
B	4	0	1	2	∞
C	8	12	0	4	2
D	6	2	3	0	7
E	∞	∞	2	7	0

recorridos:

	A	B	C	D	E
A	-	B	B	B	E
B	A	-	C	D	E
C	A	A	-	D	E
D	B	B	B	-	E
E	A	B	C	D	-



iteracion2

distancias:

	A	B	C	D	E
A	0	4	5	6	∞
B	4	0	1	2	∞
C	8	12	0	4	2
D	6	2	3	0	7
E	∞	∞	2	7	0

recorridos:

	A	B	C	D	E
A	-	B	B	B	E
B	A	-	C	D	E
C	A	A	-	D	E
D	B	B	B	-	E
E	A	B	C	D	-

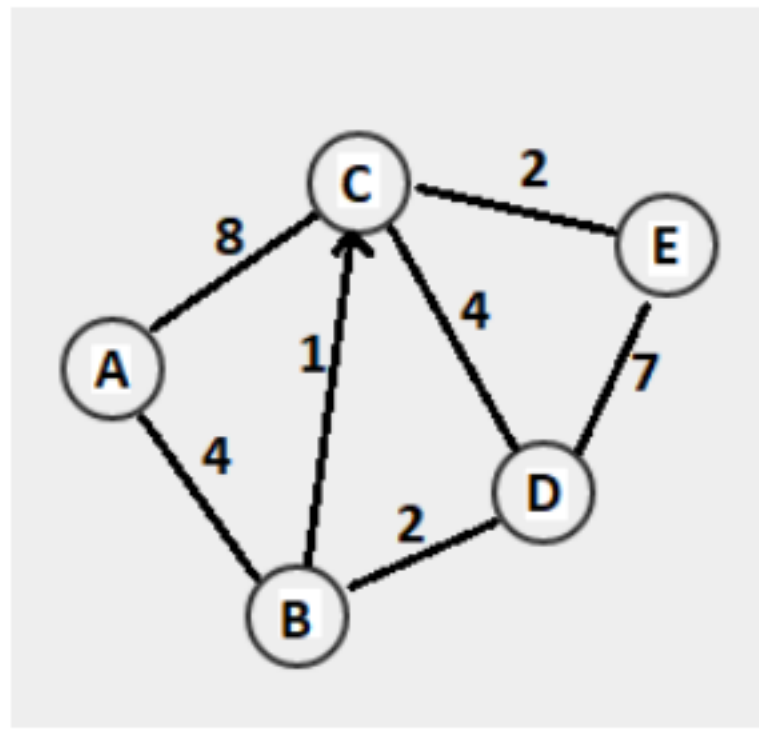
iteracion3

distancias:

	A	B	C	D	E
A	0	4	5	6	7
B	4	0	1	2	3
C	8	12	0	4	2
D	6	2	3	0	5
E	10	14	2	6	0

recorridos:

	A	B	C	D	E
A	-	B	B	B	C
B	A	-	C	D	C
C	A	A	-	D	E
D	B	B	B	-	C
E	C	C	C	C	-



iteracion3

distancias:

	A	B	C	D	E
A	0	4	5	6	7
B	4	0	1	2	3
C	8	12	0	4	2
D	6	2	3	0	5
E	10	14	2	6	0

recorridos:

	A	B	C	D	E
A	-	B	B	B	C
B	A	-	C	D	C
C	A	A	-	D	E
D	B	B	B	-	C
E	C	C	C	C	-

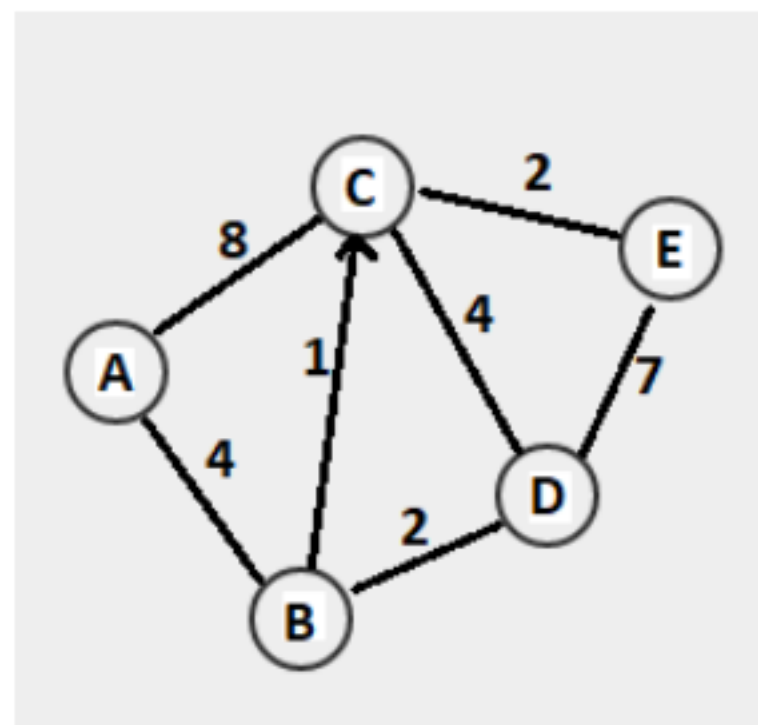
iteracion4

distancias:

	A	B	C	D	E
A	0	4	5	6	7
B	4	0	1	2	3
C	8	6	0	4	2
D	6	2	3	0	5
E	10	8	2	6	0

recorridos:

	A	B	C	D	E
A	-	B	B	B	C
B	A	-	C	D	C
C	A	D	-	D	E
D	B	B	B	-	C
E	C	D	C	C	-



iteracion4

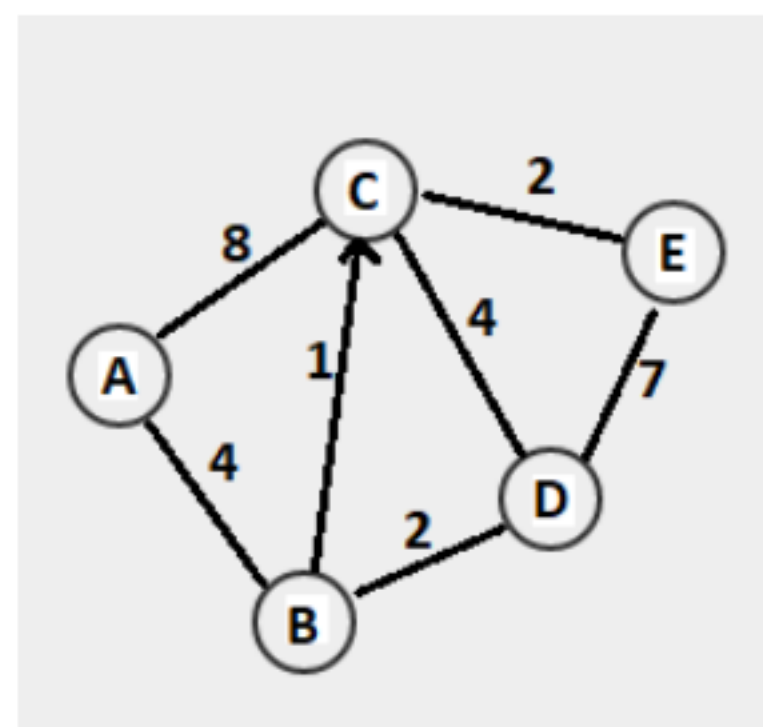
distancias:

	A	B	C	D	E
A	0	4	5	6	7
B	4	0	1	2	3
C	8	6	0	4	2
D	6	2	3	0	5
E	10	8	2	6	0

recorridos:

	A	B	C	D	E
A	-	B	B	B	C
B	A	-	C	D	C
C	A	D	-	D	E
D	B	B	B	-	C
E	C	D	C	C	-

- Para leer el recorrido de A a E:



iteracion4

distancias:

	A	B	C	D	E
A	0	4	5	6	7
B	4	0	1	2	3
C	8	6	0	4	2
D	6	2	3	0	5
E	10	8	2	6	0

recorridos:

	A	B	C	D	E
A	-	B	B	B	C
B	A	-	C	D	C
C	A	D	-	D	E
D	B	B	B	-	C
E	C	D	C	C	-

```
vector< vector<int> > ady;

vector< vector<int> > Grafo :: floydWarshall(){ vector< vector<int>
    > distancias = this->ady;
for(int i = 0; i < n; i++) //n: cantidad de nodos
    distancias[i][i] = 0;
for(int k = 0; k < n; k++)
    for(int i = 0; i < n; i++)
        for(int j = 0; j < n; j++){

            int dt = distancias[i][k] + distancias[k][j]; if(dt <
                distancias[i][j])
                distancias[i][j] = dt;    }

return distancias;}
```

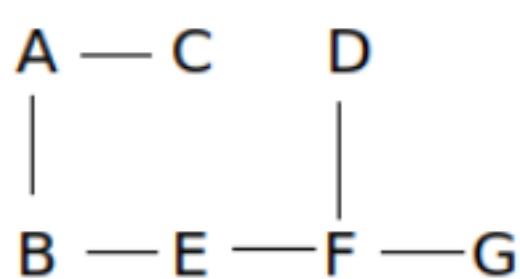

A*Algoritmo de búsqueda de tipo heurístico.

Encuentra un camino de bajo costo entre 2 pares de vértices cualesquiera pero no asegura que este sea el camino mínimo.

Al igual que Dijkstra, en cada iteración "visita" un nodo, pero a diferencia de este, no decide cual visitar en base al costo acumulado, si no que analiza que tan "prometedor" es un vértice utilizando una heurística.

Se dice aproximadamente ya que nada nos asegura que el resultado de la heurística nos lleve a resolver el problema de manera óptima, a veces ni siquiera podemos resolverlo.

Algoritmo Supongamos que tenemos este grafo:



utilizamos como heurística la distancia Manhattan, que dice que dados dos vértices con coordenadas (x, y) e (x', y') , podemos calcular su distancia como $|x - x'| + |y - y'|$

Queremos encontrar un camino entre A y D.

Para encontrarlo vamos a hacer uso de 2 estructuras S_a y S_c , sets abierto y cerrado. Ambos van a contener un subconjunto de vértices.

S_a (set abierto) va a contener los vértices conocidos pero no visitados.

S_c (set cerrado) va a contener los vértices conocidos y ya visitados.

Inicialmente $S_a = \{A\}$ y $S_c = \{\}$

Paso 1: De los vértices de S_a tomamos el más prometedor (calculado con la heurística).

Paso 2: En caso de que el vértice prometedor sea el destino, terminamos el algoritmo, si no:

Incluimos los vecinos del vértice prometedor a S_a , siempre y cuando:

- a) No esten ya en S_a
- b) No esten en S_c

Cada vértice agregado a S_a recuerda que fue el vértice prometedor quien los agregó (supadre) y el costo de llegar desde el origen hasta ellos.

En el caso de A se compara el costo que recuerda el vecino contra el costo de llegar desde el origen hasta él a través del vértice prometedor; si este último es menor, entonces el vértice prometedor pasa a ser supadre.

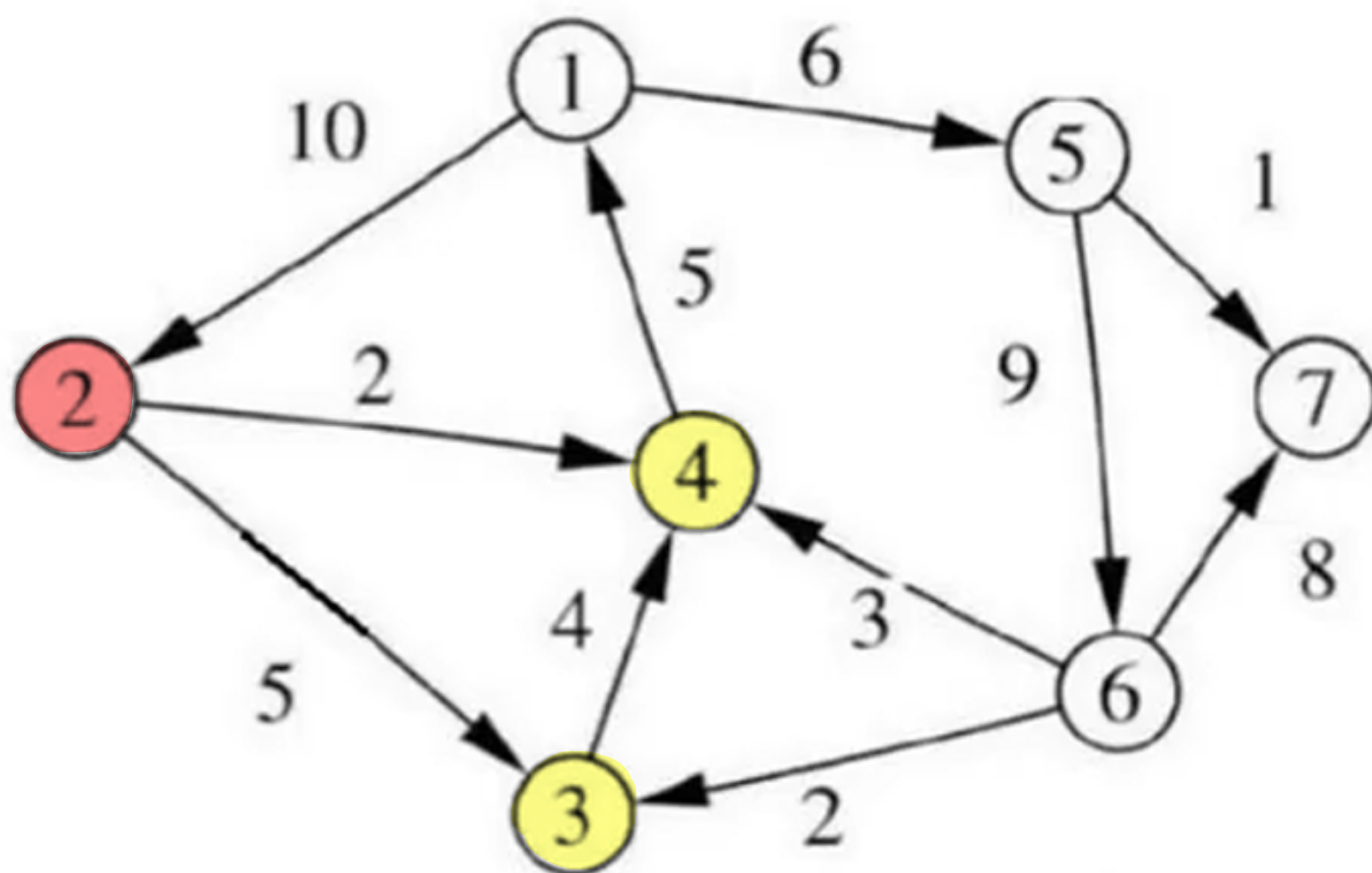
Paso 3: Sacamos el v rtice prometedor de S_a y lo agregamos a S_c .

Si S_a est  vac o, entonces terminamos el algoritmo, si no volvemos al paso 1.

Ejercicio caminos m nimos Dado el siguiente grafo, calcule paso a paso los caminos m nimos utilizando los algoritmos de Dijkstra y Floyd-Warshall, mostrando en cada paso el estado de las estructuras utilizadas.

Dijkstra

Inicializaci n: $D: \infty \ 0 \ \infty \ \infty \ \infty \ \infty \ \infty$

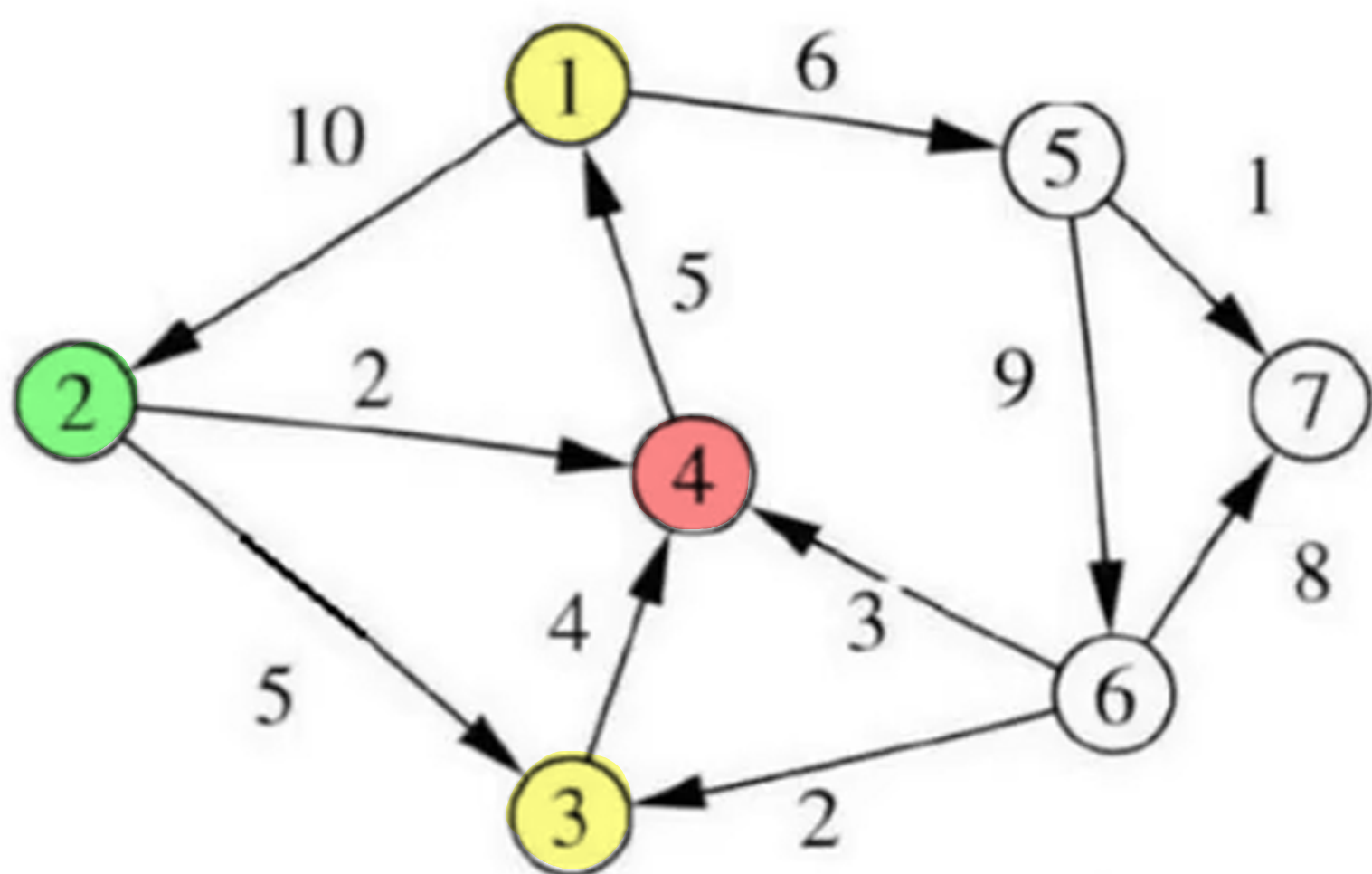


D

∞	0	∞	∞	∞	∞	∞
1	2	3	4	5	6	7

P

N	N	Z	Z	N	N	N
---	---	---	---	---	---	---

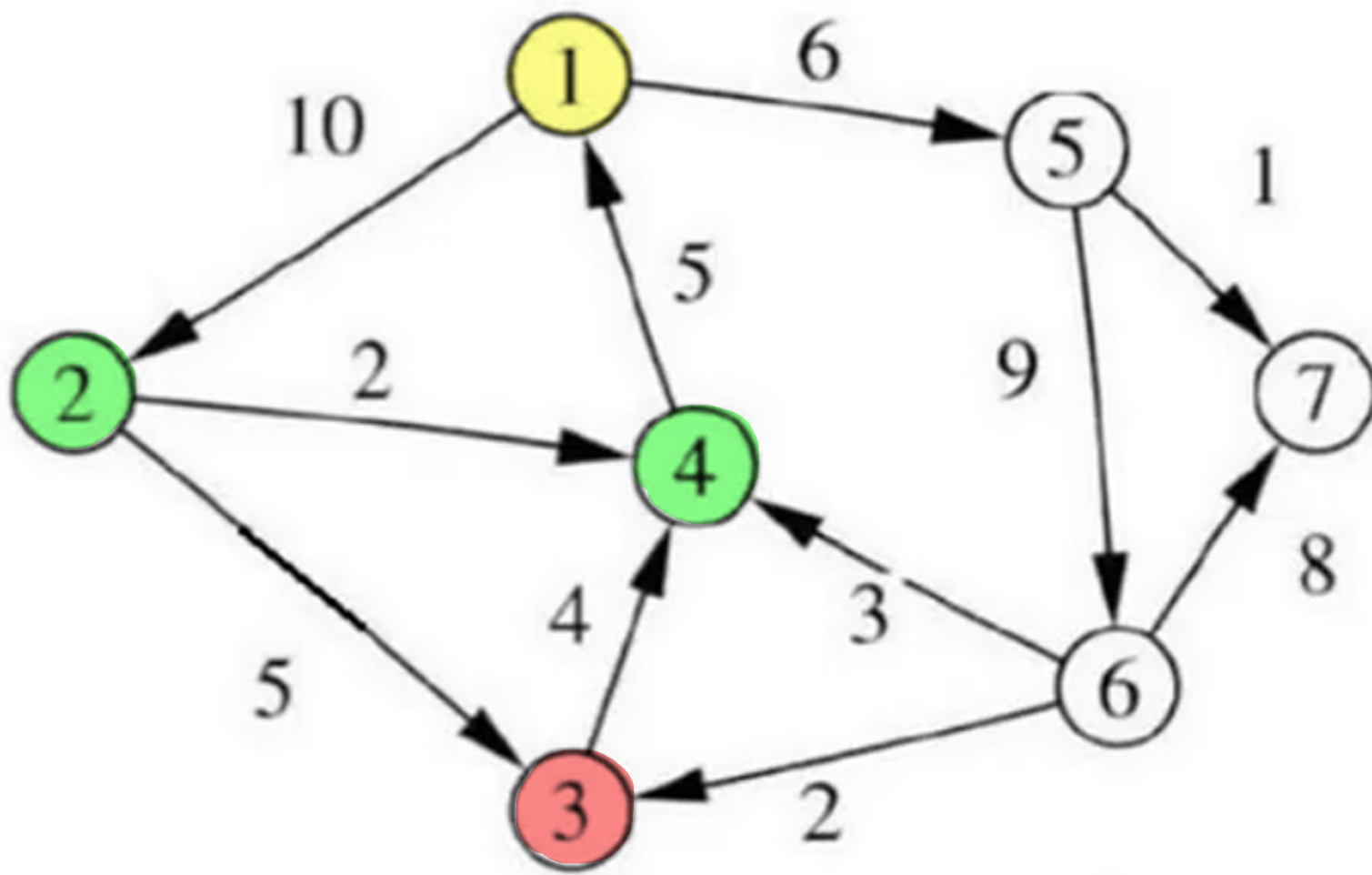


D

7	0	5	2	∞	∞	∞
1	2	3	4	5	6	7

P

4	N	Z	Z	N	N	N
---	---	---	---	---	---	---

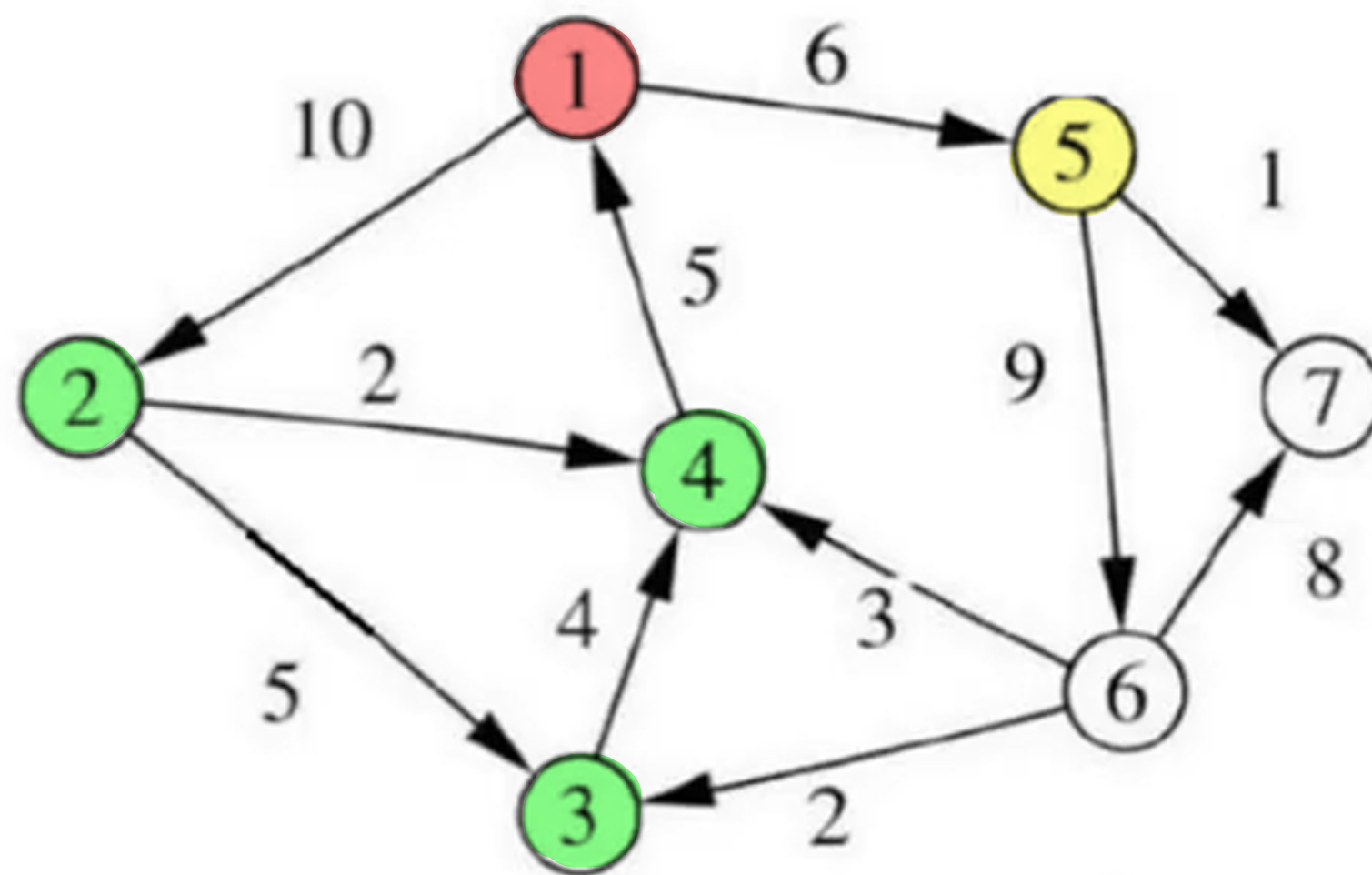


$$D$$

7	0	5	2	∞	∞	∞
1	2	3	4	5	6	7

$$P$$

4	N	2	2	N	N	N
---	---	---	---	---	---	---

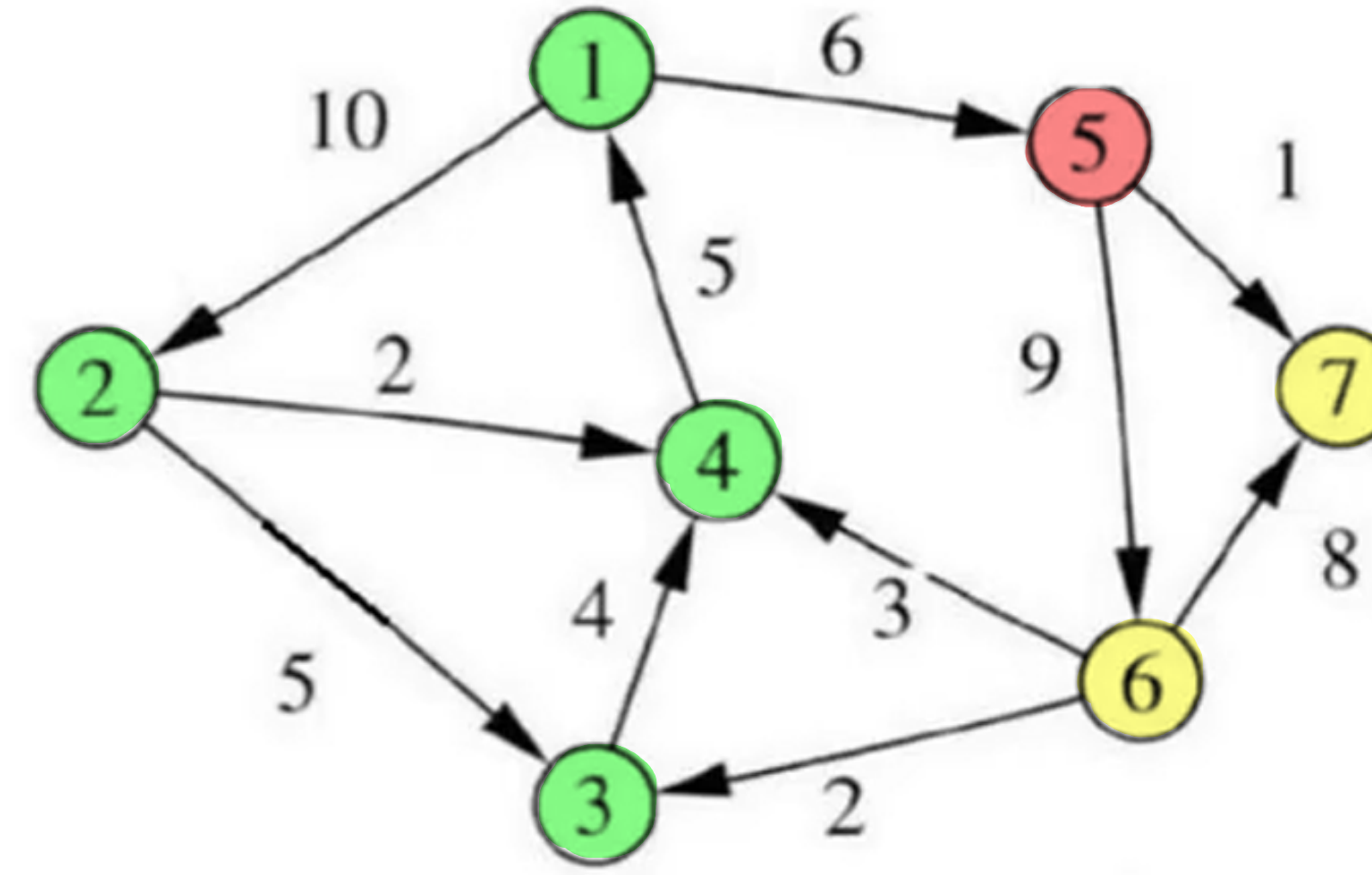


$$D$$

7	0	5	2	13	∞	∞
1	2	3	4	5	6	7

$$P$$

4	N	2	2	1	N	N
---	---	---	---	---	---	---

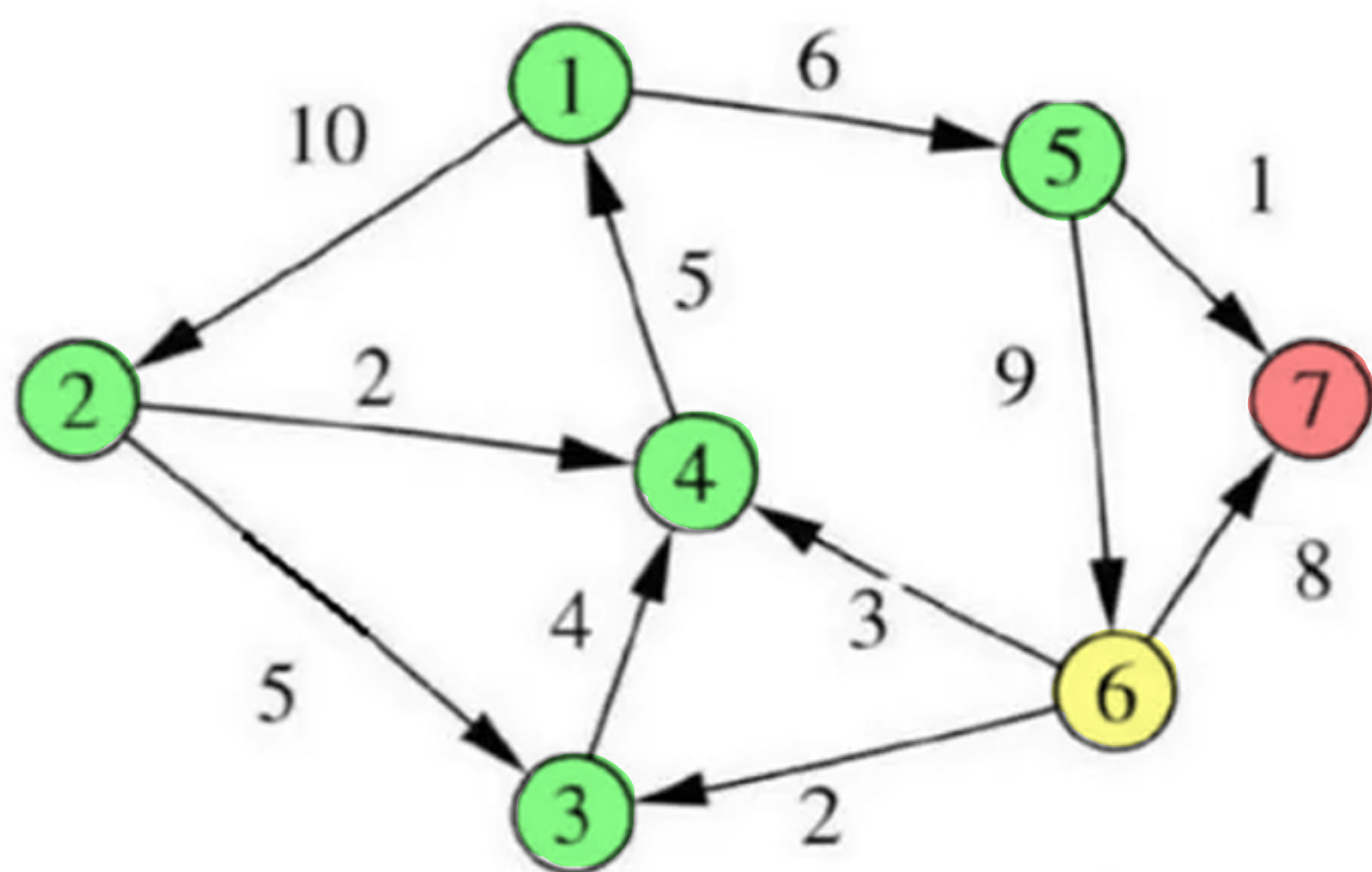


$$D$$

7	0	5	2	13	22	14
1	2	3	4	5	6	7

$$P$$

4	N	2	2	1	5	5
---	---	---	---	---	---	---

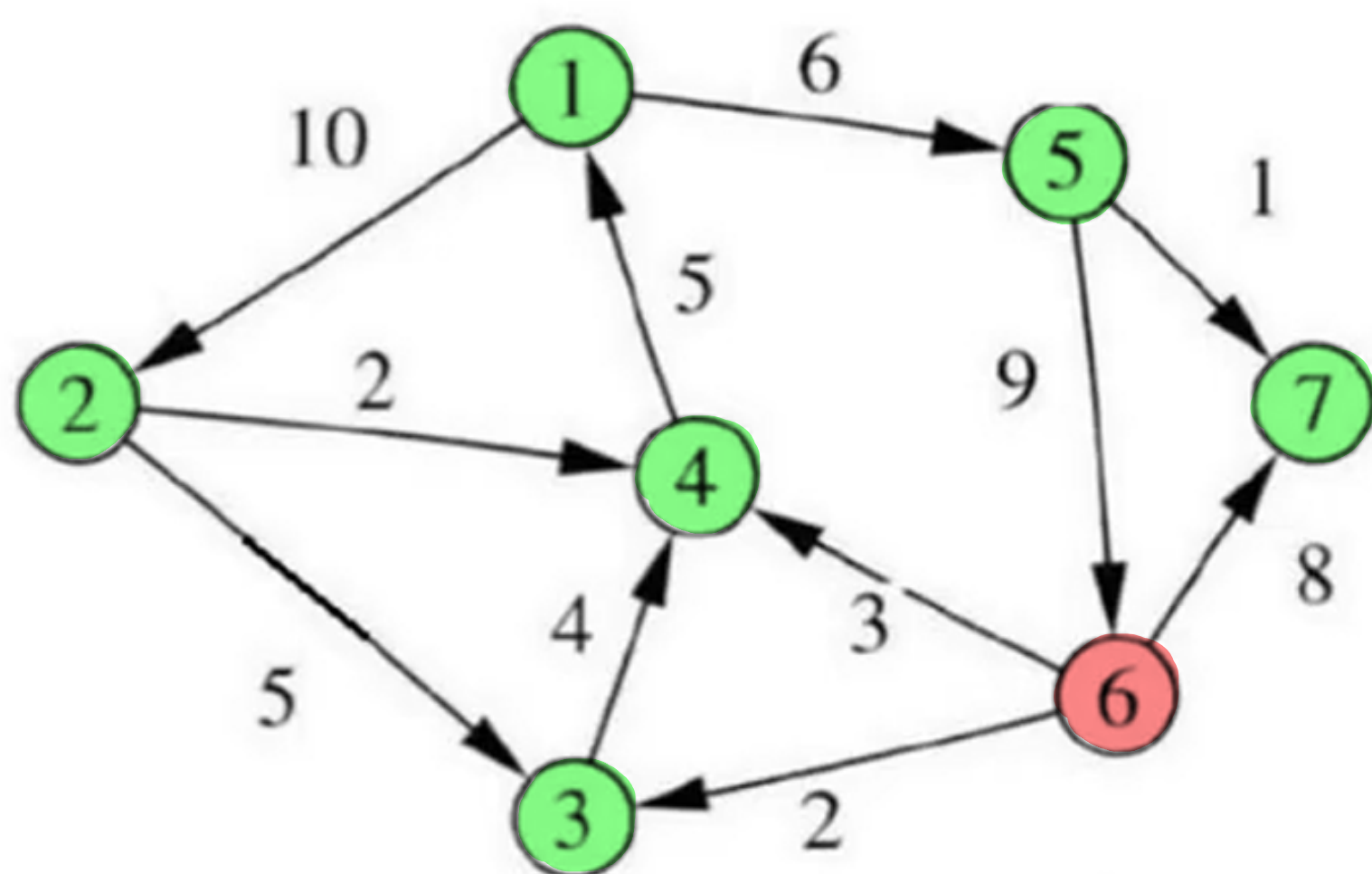


D

7	0	5	2	13	22	14
1	2	3	4	5	6	7

P

4	N	2	2	1	5	5
---	---	---	---	---	---	---

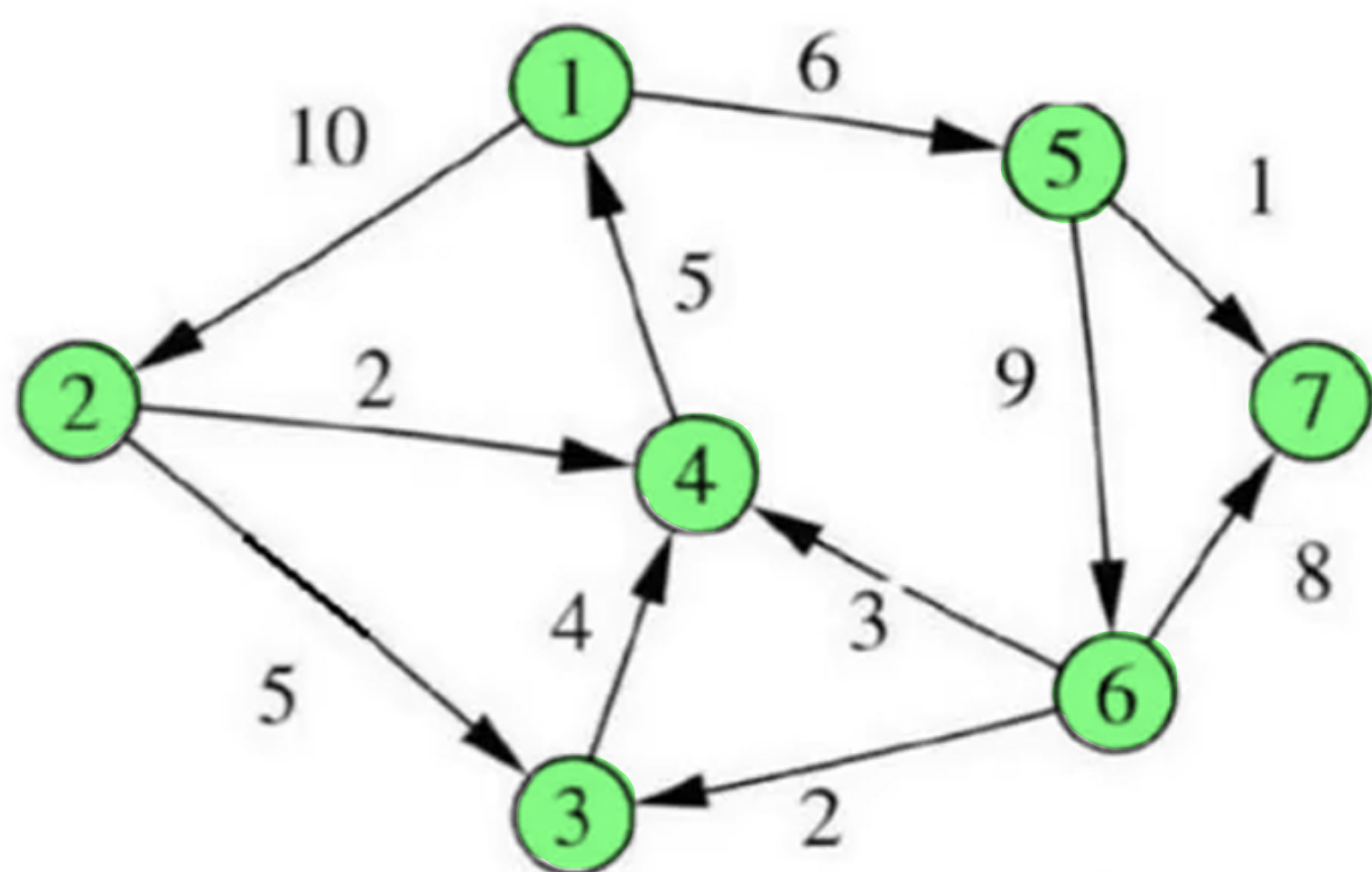


D

7	0	5	2	13	22	14
1	2	3	4	5	6	7

P

4	N	2	2	1	5	5
---	---	---	---	---	---	---



D

7	0	5	2	13	22	14
1	2	3	4	5	6	7

P

4	N	2	2	1	5	5
---	---	---	---	---	---	---

Floyd - Warshall

Inicialización

D:	1	2	3	4	5	6	7
1	0	10	∞	∞	6	∞	∞
2	∞	0	5	2	∞	∞	∞
3	∞	∞	0	4	∞	∞	∞
4	5	∞	∞	0	∞	∞	∞
5	∞	∞	∞	∞	0	9	1
6	∞	∞	2	3	∞	0	8
7	∞	∞	∞	∞	∞	∞	0

R:	1	2	3	4	5	6	7
1		1			1		
2			2	2			
3				3			
4	4						
5						5	5
6			6	6			6
7							

Iteración k=1

D:	1	2	3	4	5	6	7
1	0	10	∞	∞	6	∞	∞
2	∞	0	5	2	∞	∞	∞
3	∞	∞	0	4	∞	∞	∞
4	5	15	∞	0	11	∞	∞
5	∞	∞	∞	∞	0	9	1
6	∞	∞	2	3	∞	0	8
7	∞	∞	∞	∞	∞	∞	0

R:	1	2	3	4	5	6	7
1		1			1		
2			2	2			
3				3			
4	4	1			1		
5						5	5
6			6	6			6
7							

k=2

D:	1	2	3	4	5	6	7
1	0	10	15	12	6	∞	∞
2	∞	0	5	2	∞	∞	∞
3	∞	∞	0	4	∞	∞	∞
4	5	15	20	0	11	∞	∞
5	∞	∞	∞	∞	0	9	1
6	∞	∞	2	3	∞	0	8
7	∞	∞	∞	∞	∞	∞	0

R:	1	2	3	4	5	6	7
1		1	2	2	1		
2			2	2			
3				3			
4	4	1	2		1		
5						5	5
6			6	6			6
7							

k=3

D:	1	2	3	4	5	6	7
1	0	10	15	12	6	∞	∞
2	∞	0	5	2	∞	∞	∞
3	∞	∞	0	4	∞	∞	∞
4	5	15	20	0	11	∞	∞
5	∞	∞	∞	∞	0	9	1
6	∞	∞	2	3	∞	0	8
7	∞	∞	∞	∞	∞	∞	0

R:	1	2	3	4	5	6	7
1		1	2	2	1		
2			2	2			
3				3			
4	4	1	2		1		
5						5	5
6			6	6			6
7							

k=4

D:	1	2	3	4	5	6	7
1	0	10	15	12	6	∞	∞
2	7	0	5	2	13	∞	∞
3	9	19	0	4	15	∞	∞
4	5	15	20	0	11	∞	∞
5	∞	∞	∞	∞	0	9	1
6	8	18	2	3	14	0	8
7	∞	∞	∞	∞	∞	∞	0

R:	1	2	3	4	5	6	7
1		1	2	2	1		
2	1		2	2	1		
3	4	1		3	1		
4	4	1	2		1		
5						5	5
6	4	1	6	6	1		6
7							

$$R[i][j] = R[k][j]$$

k=5

D:	1	2	3	4	5	6	7
1	0	10	15	12	6	15	7
2	7	0	5	2	13	22	14
3	9	19	0	4	15	24	16
4	5	15	20	0	11	20	12
5	∞	∞	∞	∞	0	9	1
6	8	18	2	3	14	0	8
7	∞	∞	∞	∞	∞	∞	0

R:	1	2	3	4	5	6	7
1		1	2	2	1	5	5
2	1		2	2	1	5	5
3	4	1		3	1	5	5
4	4	1	2		1	5	5
5						5	5
6	4	1	6	6	1		6
7							

$$R[i][j] = R[k][j]$$

k = 6

D:	1	2	3	4	5	6	7
1	0	10	15	12	6	15	7
2	7	0	5	2	13	22	14
3	9	19	0	4	15	24	16
4	5	15	20	0	11	20	12
5	17	27	11	12	0	9	1
6	8	18	2	3	14	0	8
7	∞	∞	∞	∞	∞	∞	0

k = 7

D:	1	2	3	4	5	6	7
1	0	10	15	12	6	15	7
2	7	0	5	2	13	22	14
3	9	19	0	4	15	24	16
4	5	15	20	0	11	20	12
5	17	27	11	12	0	9	1
6	8	18	2	3	14	0	8
7	∞	∞	∞	∞	∞	∞	0

$$R[i][j] = R[k][j]$$

R:	1	2	3	4	5	6	7
1		1	2	2	1	5	5
2	1		2	2	1	5	5
3	4	1		3	1	5	5
4	4	1	2		1	5	5
5	4	1	6	6		5	5
6	4	1	6	6	1		6
7							

$$R[i][j] = R[k][j]$$

R:	1	2	3	4	5	6	7
1		1	2	2	1	5	5
2	1		2	2	1	5	5
3	4	1		3	1	5	5
4	4	1	2		1	5	5
5	4	1	6	6		5	5
6	4	1	6	6	1		6
7							